

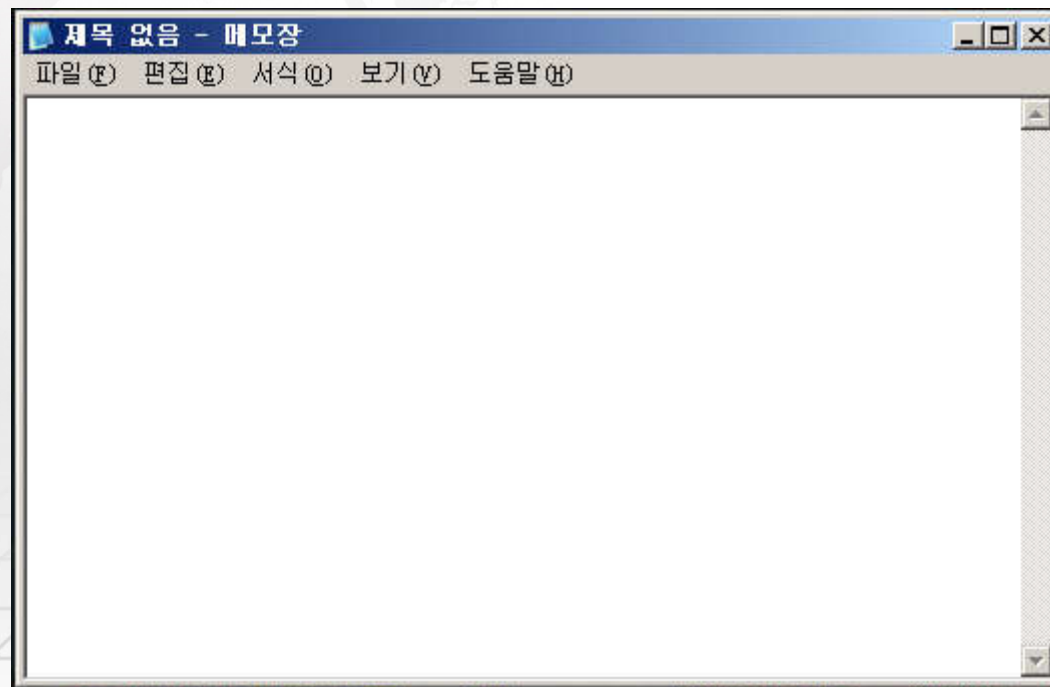
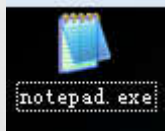
PE文件基础

- ◆什么是PE文件？
- ◆PE文件的全称是Portable Executable，意为可移植的可执行文件，常见的EXE、DLL、OCX、SYS、COM都是PE文件，PE文件是微软Windows操作系统上的程序文件（可能是间接被执行，如DLL）。

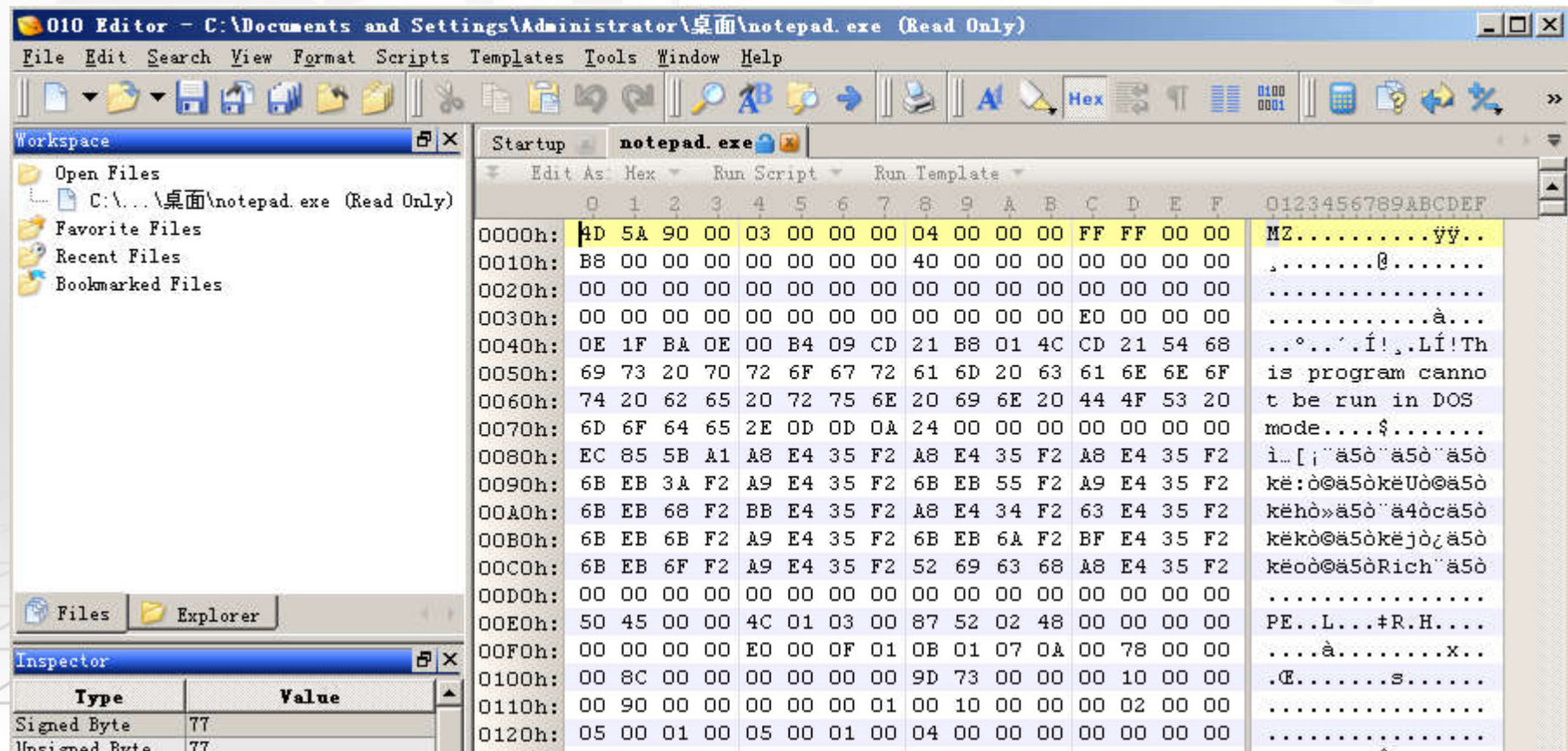
一、为什么要重点学习这种文件格式呢？

- ◆PE文件是可移植、可执行、跨Win32平台的文件格式
所有Win32执行体（exe、dll、kernel mode drivers）
- ◆知道PE文件本质后，能更好分析恶意样本、APT攻击分析、勒索病毒分析
- ◆了解软件加密和加壳的思想，能够PI相关的PE文件
- ◆它是您熟悉Windows操作系统的第一步，包括EXE程序怎么映射到内存，DLL怎么导入等
- ◆软件逆向工程的基本思想与PE文件格式息息相关
- ◆如果您想成为一名黑客、系统安全工程师，那么精通PE文件是非常必要的

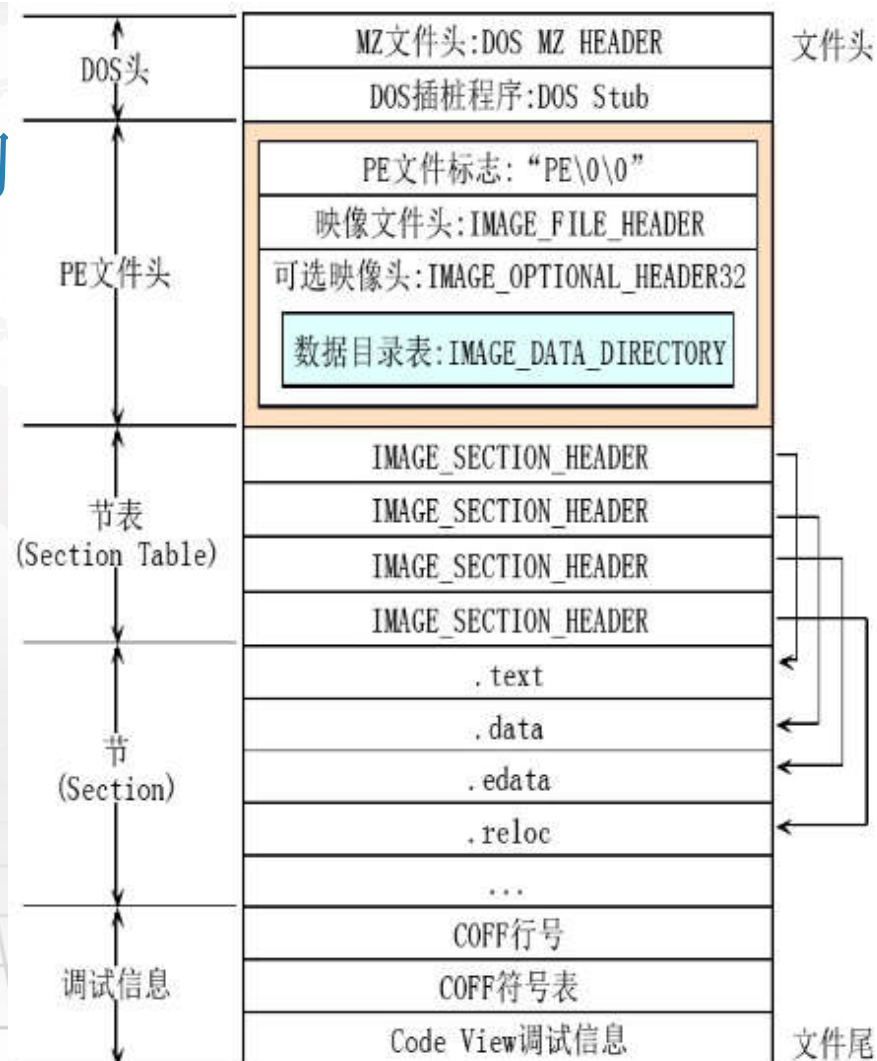
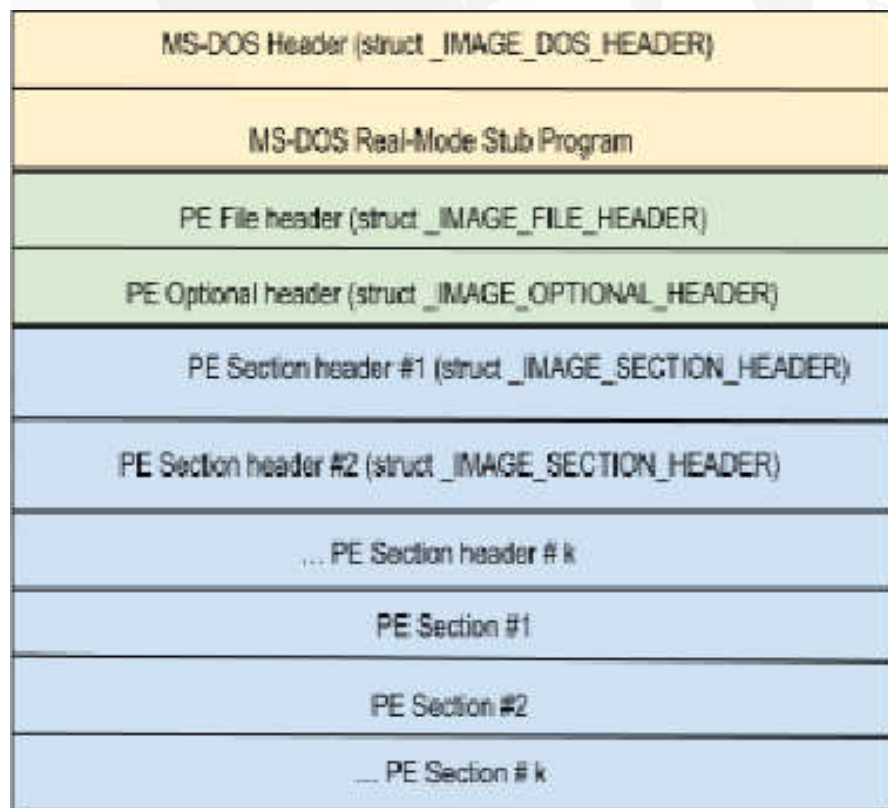
可执行程序是具有不同的形态的，比如用户眼中的记事本如下图所示。



在逆向工程师眼中，它是这样的。



PE文件格式总体结构



PE文件解析常用工具

包括：

- **PEView**：可按照PE文件格式对目标文件的各字段进行详细解析。
- **Stud_PE**：可按照PE文件格式对目标文件的各字段进行详细解析。
- **Ollydbg**：可跟踪目标程序的执行过程，属于用户态调试工具。
- **UltraEdit \ 010Editor**：可对目标文件进行16进制查看和修改。

使用O10Editor工具打开PE文件，并运行模板

Startup notepad.exe

Edit As: Hex Run Script Run Template: PE.bt

0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF

0000h: 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 MZ.....yy..
0010h: B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00@.....
0020h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0030h: 00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00à...
0040h: 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 ..°..'.í!..Lí!Th
0050h: 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F is program canno
0060h: 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 t be run in DOS
0070h: 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00 mode....\$.....
0080h: EC 85 5B A1 A8 E4 35 F2 A8 E4 35 F2 A8 E4 35 F2 i...;`a50`a50`a50
0090h: 6B EB 3A F2 A9 E4 35 F2 6B EB 55 F2 A9 E4 35 F2 kē:ò@a50kēUò@a50
00A0h: 6B EB 68 F2 BB E4 35 F2 A8 E4 34 F2 63 E4 35 F2 kēhò»a50`a4òcà50
00B0h: 6B EB 6B F2 A9 E4 35 F2 6B EB 6A F2 BF E4 35 F2 kēkò@a50kējòçà50
00C0h: 6B EB 6F F2 A9 E4 35 F2 52 69 63 68 A8 E4 35 F2 kēoò@a50Rich`a50
00D0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Template Results - PE.bt

Name	Value	Start	Size	Color	Comment
struct IMAGE_DOS_HEADER DosHeader		0h	40h	Fg: Bg:	
WORD MZSignature	5A4Dh				
WORD UsedBytesInTheLastPage	144	00000000	4D 5A	90 00 03 00 00 00 00 04 00 00 00 FF FF 00 00	MZ yy
WORD FileSizeInPages	3	00000010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	.	@
WORD NumberOfRelocationItems	0	00000020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		
WORD HeaderSizeInParagraphs	4	00000030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		à
WORD MinimumExtraParagraphs	0				
WORD MaximumExtraParagraphs	65535				
WORD InitialRelativeSS	0	Eh	2h	Fg: Bg:	Initial (relative) SS value
WORD InitialSP	184	10h	2h	Fg: Bg:	Initial SP value
WORD Checksum	0	12h	2h	Fg: Bg:	Checksum
WORD InitialIP	0	14h	2h	Fg: Bg:	Initial IP value
WORD InitialRelativeCS	0	16h	2h	Fg: Bg:	Initial (relative) CS value
WORD AddressOfRelocationTable	64	18h	2h	Fg: Bg:	File address of relocation table
WORD OverlayNumber	0	1Ah	2h	Fg: Bg:	Overlay number

NT头

NT头IMAGE NT HEADERS由三个成员组成，大小为F8

00B0h: 6B EB 6B F2 A9 E4 35 F2 6B EB 6A F2 BF E4 35 F2
00C0h: 6B EB 6F F2 A9 E4 35 F2 52 69 63 68 A8 E4 35 F2
00D0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00E0h: 50 45 00 00 4C 01 03 00 87 52 02 48 00 00 00 00
00F0h: 00 00 00 00 E0 00 0F 01 0B 01 07 0A 00 78 00 00
0100h: 00 8C 00 00 00 00 00 00 9D 73 00 00 00 10 00 00
0110h: 00 90 00 00 00 00 00 01 00 10 00 00 00 02 00 00
0120h: 05 00 01 00 05 00 01 00 04 00 00 00 00 00 00 00
0130h: 00 40 01 00 00 04 00 00 CE 26 01 00 02 00 00 80
0140h: 00 00 04 00 00 10 01 00 00 00 10 00 00 10 00 00
0150h: 00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00
0160h: 04 76 00 00 C8 00 00 00 00 B0 00 00 04 83 00 00
0170h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0180h: 00 00 00 00 00 00 00 00 50 13 00 00 1C 00 00 00
0190h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
01A0h: 00 00 00 00 00 00 00 00 A8 18 00 00 40 00 00 00
01B0h: 50 02 00 00 D0 00 00 00 00 10 00 00 48 03 00 00
01C0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
01D0h: 00 00 00 00 00 00 00 00 2E 74 65 78 74 00 00 00
01E0h: 48 77 00 00 00 10 00 00 00 78 00 00 00 04 00 00
01F0h: 00 00 00 00 00 00 00 00 00 00 00 00 20 00 00 60

kekò@ä5òkējòçä5ò
kēoò@ä5òRich"ä5ò
.....
PE..L...+R.H...
....à.....x..
.Œ.....s.....
.....
.0.....î&.....€
.....
.v..È....°...f..
.....
.....P.....
.....
P...Đ.....H..
.....
......text...
Hw.....x.....
.....`

PE文件头

PE文件标志: "PE\0\0"

映像文件头: IMAGE_FILE_HEADER

可选映像头: IMAGE_OPTIONAL_HEADER32

数据目录表: IMAGE_DATA_DIRECTORY

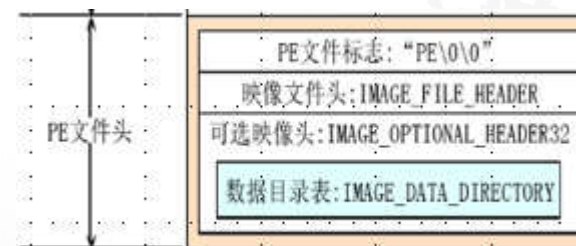
Template Results - PE.bt

Name	Value	Start	Size	Color
☒ struct IMAGE_DOS_HEADER DosHeader		0h	40h	Fg: Bg:
☒ struct IMAGE_DOS_STUB DosStub		40h	90h	Fg: Bg:
☒ struct IMAGE_NT_HEADERS NtHeader		E0h	F8h	Fg: Bg:
☒ DWORD Signature	4550h	F0h	4h	Fg: Bg:

NT头

NT头IMAGE_NT_HEADERS由三个成员组成，大小为F8

- 第一个成员：签名（Signature）结构体
- Signature从0xE0开始



000000E0 50 45 00 00 4C 01 03 00 87 52 02 48 00 00 00 00 PE L #R H

NT头

第二个成员：文件头（File Header）File Header为表现文件大致属性的文件头，IMAGE_FILE_HEADER结构体，重要成员：

pFile	Data	Description	Value
000000E4	014C	Machine	IMAGE_FILE_MACHINE_I386
000000E6	0003	Number of Sections	
000000EC	00000000	Pointer to Symbol Table	
000000F0	00000000	Number of Symbols	
000000F4	00E0	Size of Optional Header	
000000F6	010F	Characteristics	
	0001		IMAGE_FILE_RELOCS_STRIPPED
	0002		IMAGE_FILE_EXECUTABLE_IMAGE
	0004		IMAGE_FILE_LINE_NUMS_STRIPPED
	0008		IMAGE_FILE_LOCAL_SYMS_STRIPPED
	0100		IMAGE_FILE_32BIT_MACHINE

PE文件头

PE文件标志: "PE\0\0"

映像文件头: IMAGE_FILE_HEADER

可选映像头: IMAGE_OPTIONAL_HEADER32

数据目录表: IMAGE_DATA_DIRECTORY

- 1、Machine: 标识兼容的CPU, 本程序中值为0x014C, 兼容Intel386
- 2、NumberOfSection: 用于指出文件中存在的节区数量, 该值一定要大于0, 且当定义的节区数量与实际节区不同时, 将发生运行错误, 本程序中值为0x0003
- 3、SizeOfOptionalHeader: 用于指出最后一个NT头成员 (可选头) 的大小, 本程序中值为0x00E0
- 4、Characteristics: 用于标识文件的属性, 文件是否是可运行的形态, 是否为DLL文件等信息, 本程序中值为0x010F

000000E0	50 45 00 00 4C 01 03 00	87 52 02 48 00 00 00 00	PE	L	#R H
000000F0	00 00 00 00 E0 00 0F 01	0B 01 07 0A 00 78 00 00		à	x

000000F6	010F	Characteristics
	0001	IMAGE_FILE_RELOCS_STRIPPED
	0002	IMAGE_FILE_EXECUTABLE_IMAGE
	0004	IMAGE_FILE_LINE_NUMS_STRIPPED
	0008	IMAGE_FILE_LOCAL_SYMS_STRIPPED
	0100	IMAGE_FILE_32BIT_MACHINE

第三个成员：可选头（Optional Header）

Bookmarked Files

Files Explorer

Inspector

Type	Value
Signed Byte	11
Unsigned Byte	11
Signed Short	267
Unsigned Short	267
Signed Int	168231179
Unsigned Int	168231179
Signed Int64	131941563564299
Unsigned Int64	131941563564299
Float	6.50021e-33
Double	6.51877938156993e-310
Half Float	1.591444e-05
String	
Unicode	

Auto Variables Bookmarks

00D0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00E0h: 50 45 00 00 4C 01 03 00 87 52 02 48 00 00 00 00 PE..L...+R.H...
00F0h: 00 00 00 00 E0 00 0F 01 0B 01 07 0A 00 78 00 00à.....x..
0100h: 00 8C 00 00 00 00 00 00 9D 73 00 00 00 10 00 00 .@.....s.....
0110h: 00 90 00 00 00 00 00 01 00 10 00 00 00 02 00 00
0120h: 05 00 01 00 05 00 01 00 04 00 00 00 00 00 00 00
0130h: 00 40 01 00 00 04 00 00 CE 26 01 00 02 00 00 80 .@.....î.....€
0140h: 00 00 04 00 00 10 01 00 00 00 10 00 00 10 00 00
0150h: 00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00
0160h: 04 76 00 00 C8 00 00 00 00 80 00 00 04 83 00 00 .v..È....°...f..
0170h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0180h: 00 00 00 00 00 00 00 00 50 13 00 00 1C 00 00 00P.....
0190h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
01A0h: 00 00 00 00 00 00 00 00 A8 18 00 00 40 00 00 00@..
01B0h: 50 02 00 00 D0 00 00 00 00 10 00 00 48 03 00 00 P...D.....H..
01C0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
01D0h: 00 00 00 00 00 00 00 00 2E 74 65 78 74 00 00 00text..
01E0h: 48 77 00 00 00 10 00 00 00 78 00 00 00 04 00 00 Hw.....x.....

Template Results - PE.bt

Name	Value	Start	Size
struct IMAGE_DOS_HEADER DosHeader		0h	40h
struct IMAGE_DOS_STUB DosStub		40h	90h
struct IMAGE_NT_HEADERS NtHeader		E0h	F8h
DWORD Signature	4550h	E0h	4h
struct IMAGE_FILE_HEADER FileHeader		E4h	14h
struct IMAGE_OPTIONAL_HEADER32 OptionalHeader		F8h	E0h
struct IMAGE_SECTION_HEADER SectionHeaders[3]		1D8h	78h

pFile	Data	Description	Value
000000F8	010B	Magic	IMAGE_NT_OPTIONAL_HDR32_MAGIC
000000FA	07	Major Linker Version	
000000FB	0A	Minor Linker Version	
000000FC	00007800	Size of Code	
00000100	00008C00	Size of Initialized Data	
00000104	00000000	Size of Uninitialized Data	
00000108	0000739D	Address of Entry Point	
0000010C	00001000	Base of Code	
00000110	00009000	Base of Data	
00000114	01000000	Image Base	
00000118	00001000	Section Alignment	
0000011C	00000200	File Alignment	
00000120	0005	Major O/S Version	
00000122	0001	Minor O/S Version	
00000124	0005	Major Image Version	
00000126	0001	Minor Image Version	
00000128	0004	Major Subsystem Version	
0000012A	0000	Minor Subsystem Version	
0000012C	00000000	Win32 Version Value	
00000130	00014000	Size of Image	
00000134	00000400	Size of Headers	
00000138	000126CE	Checksum	
0000013C	0002	Subsystem	IMAGE_SUBSYSTEM_WINDOWS_GUI
0000013E	8000	DLL Characteristics	IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
	8000		
00000140	00040000	Size of Stack Reserve	
00000144	00011000	Size of Stack Commit	
00000148	00100000	Size of Heap Reserve	
0000014C	00001000	Size of Heap Commit	
00000150	00000000	Loader Flags	
00000154	00000010	Number of Data Directories	
00000158	00000000	RVA	EXPORT Table
0000015C	00000000	Size	
00000160	00007604	RVA	IMPORT Table
00000164	000000C8	Size	

第三个成员：可选头（Optional Header） P57

- Optional Header为IMAGE_OPTIONAL_HEADER32结构体，重要成员：
- 1、Magic: Magic码为0x10B时，为IMAGE_OPTIONAL_HEADER32结构体，Magic码为0x20B时，为IMAGE_OPTIONAL_HEADER64结构体
- 2、AddressOfEntryPoint: 指EP的RVA值，指出程序最先执行的代码起始地址
- 3、ImageBase: 指出文件的优先装入地址，一般情况下EXE文件的ImageBase值为0x00400000，DLL文件为0x10000000，执行PE文件时，PE装载器先创建进程，再将文件载入内存，然后把EIP寄存器的值设置为ImageBase+AddressOfEntryPoint(若该地址被占用，选择其它空闲地址)
- 4、SectionAlignment, FileAlignment: FileAlignment指定了节区在磁盘文件中的最小单位，SectionAlignment指定了节区在内存中的最小单位。磁盘文件或内存的大小必为FileAlignment或SectionAlignment值的整数倍（200H,1000H）
- 5、SizeOfImage: 指定了PE Image在虚拟内存中所占空间的大小

第三个成员：可选头（Optional Header） P98

- 6、SizeOfHeaders：用来指出整个PE头的大小，该值也必须是FileAlignment的整数倍，第一节区所在位置与SizeOfHeaders距文件开始偏移的量相同
- 7、Subsystem：用于区分系统驱动文件（*.sys）与普通的可执行文件（*.exe，*.dll），值：1（Drive文件，系统驱动 eg: ntfs.sys）2（GUI文件，窗口应用程序 eg: notepad.exe）3（CUI文件，控制台应用程序 eg: cmd.exe）
- 8、NumberOfRvaAndSizes：（IMAGE_OPTIONAL_HEADER32结构体的最后一个成员）用于指定DataDirectory数组的个数
- 9、DataDirectory：由IMAGE_DATA_DIRECTORY结构体组成的数组，其中DataDirectory[0]= EXPORT Directory，DataDirectory[1] = IMPORT Directory，DataDirectory[2] = RESOURCE Directory（RVA与Size）共16个。

第三个成员：可选头（Optional Header）

P98

16个
IMAGE_DATA_DIRECTORY
Y

000000F0	00 00 00 00 E0 00 0F 01	0B 01 07 0A 00 78 00 00	à	x
00000100	00 8C 00 00 00 00 00 00	9D 73 00 00 00 10 00 00	Œ	s
00000110	00 90 00 00 00 00 00 01	00 10 00 00 00 02 00 00		
00000120	05 00 01 00 05 00 01 00	04 00 00 00 00 00 00 00		
00000130	00 40 01 00 00 04 00 00	CE 26 01 00 02 00 00 80	@	î& €
00000140	00 00 04 00 00 10 01 00	00 00 10 00 00 10 00 00		
00000150	00 00 00 00 10 00 00 00	00 00 00 00 00 00 00 00		
00000160	04 76 00 00 C8 00 00 00	00 B0 00 00 04 83 00 00	v È ° f	
00000170	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
00000180	00 00 00 00 00 00 00 00	50 13 00 00 1C 00 00 00		P
00000190	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000001A0	00 00 00 00 00 00 00 00	A8 18 00 00 40 00 00 00		- @
000001B0	50 02 00 00 D0 00 00 00	00 10 00 00 48 03 00 00	P Æ	H
000001C0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000001D0	00 00 00 00 00 00 00 00	2E 74 65 78 74 00 00 00		.text

几个常用术语

- **RVA : Relative Virtual Address**，表示相对虚拟地址。它是相对内存中**ImageBase**的偏移位置。
- **VA:Virtual Address**，表示虚拟地址，即装载到内存中之后的位置。 $(VA=ImageBase+RVA)$ 。
- 举例说明，如果PE文件装入内存空间的400000h处，且进程从虚拟地址401000h开始执行，我们可以说进程执行起始地址在RVA 1000h。每个RVA都是相对于模块的起始VA的。

几个常用术语-对齐

- 内存中节对齐的粒度。文件中节对齐的粒度问题1:桶的容量为100升, 现有367升水, 请问需要使用多少个桶?
- 问题2:代码节的代码实际长度为0x46字节, 文件中节对齐粒度为0x200.内存中节对齐粒度为0x1000字节, 请问代码节在文件和内存中分别占用多少字节?。
- 为什么PE文件中有很多“00”字节?

PEview - C:\Documents and Settings\Administrator\桌面\notepad.exe

牛(F) 视图(V) 前往(G) 帮助(H)



notepad.exe

- IMAGE_DOS_HEADER
- MS-DOS Stub Program
- IMAGE_NT_HEADERS
 - Signature
 - IMAGE_FILE_HEADER
 - IMAGE_OPTIONAL_HEADER
- IMAGE_SECTION_HEADER .text
- IMAGE_SECTION_HEADER .data
- IMAGE_SECTION_HEADER .rsrc
- BOUND_IMPORT Directory Table
- BOUND_IMPORT DLL Names
- SECTION .text
- SECTION .data
- SECTION .rsrc

pFile	Data	Description	Value
000000F8	010B	Magic	IMAGE_NT_OPTIONAL_HDR32_MAGIC
000000FA	07	Major Linker Version	
000000FB	0A	Minor Linker Version	
000000FC	00007800	Size of Code	
00000100	00008C00	Size of Initialized Data	
00000104	00000000	Size of Uninitialized Data	
00000108	0000739D	Address of Entry Point	
0000010C	00001000	Base of Code	
00000110	00009000	Base of Data	
00000114	01000000	Image Base	
00000118	00001000	Section Alignment	
0000011C	00000200	File Alignment	
00000120	0005	Major O/S Version	
00000122	0001	Minor O/S Version	
00000124	0005	Major Image Version	
00000126	0001	Minor Image Version	
00000128	0004	Major Subsystem Version	
0000012A	0000	Minor Subsystem Version	
0000012C	00000000	Win32 Version Value	
00000130	00014000	Size of Image	
00000134	00000400	Size of Headers	
00000138	000126CE	Checksum	
0000013C	0002	Subsystem	IMAGE_SUBSYSTEM_WINDOWS_GUI
0000013E	8000	DLL Characteristics	IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
	8000		
00000140	00040000	Size of Stack Reserve	
00000144	00011000	Size of Stack Commit	
00000148	00100000	Size of Heap Reserve	
0000014C	00010000	Size of Heap Commit	

DataDirectory数据目录

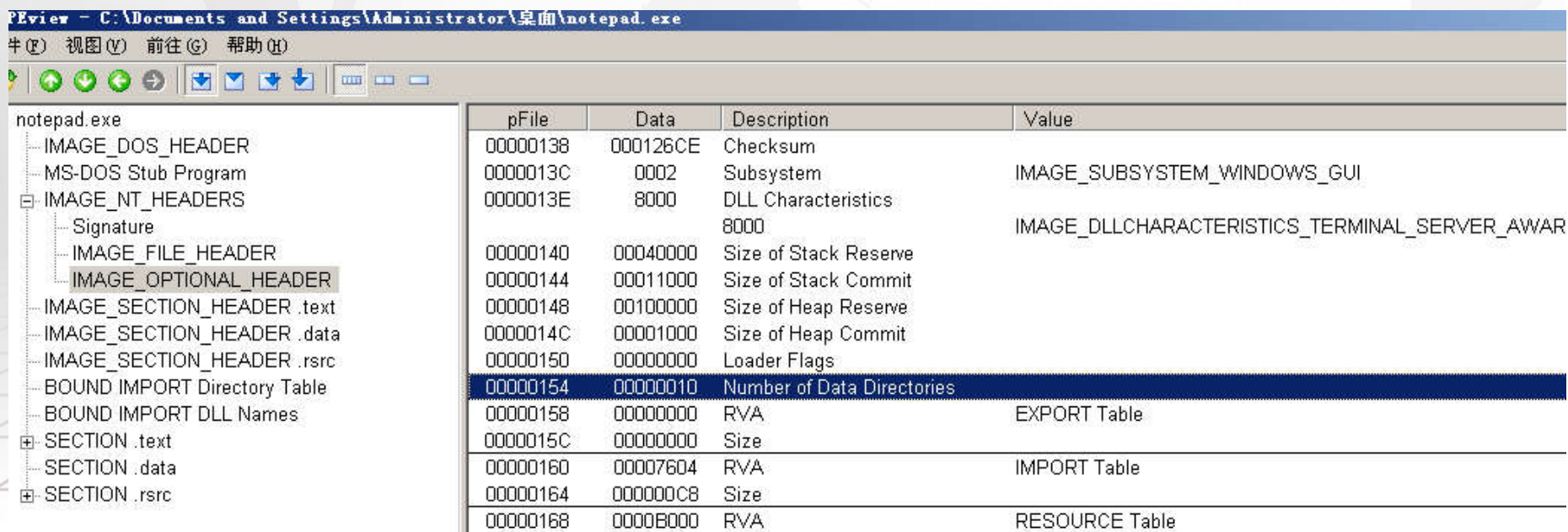
- 一个IMAGE_DATA_DIRECTORY数组,里面放的是这个可执行文件的一些重要部分的RVA和尺寸,目的是使可执行文件的装入更快,数组的项数由上一个域给出。

代码13-6 IMAGE_OPTIONAL_HEADER32结构体

```
typedef struct _IMAGE_DATA_DIRECTORY {  
    DWORD    VirtualAddress;  
    DWORD    Size;  
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;  
  
#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES    16
```

DataDirectory数据目录

- 一个IMAGE_DATA_DIRECTORY数组,里面放的是这个可执行文件的一些重要部分的RVA和尺寸,目的是使可执行文件的装入更快,数组的项数由上一个域给出。



pFile	Data	Description	Value
00000138	000126CE	Checksum	
0000013C	0002	Subsystem	IMAGE_SUBSYSTEM_WINDOWS_GUI
0000013E	8000	DLL Characteristics	
	8000		IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWAR
00000140	00040000	Size of Stack Reserve	
00000144	00011000	Size of Stack Commit	
00000148	00100000	Size of Heap Reserve	
0000014C	00001000	Size of Heap Commit	
00000150	00000000	Loader Flags	
00000154	00000010	Number of Data Directories	
00000158	00000000	RVA	EXPORT Table
0000015C	00000000	Size	
00000160	00007604	RVA	IMPORT Table
00000164	000000C8	Size	
00000168	0000B000	RVA	RESOURCE Table

节区头（节表）

- 节表其实就是紧挨着PE header的一结构数组。
- 该数组成员的数目由File header(IMAGE_FILE_HEADER)结构中NumberOfSections域的域值来决定。
- 。节表结构又命名为IMAGE_SECTION_HEADER（每个结构占用28H）。
- 结构中放的是一个节的信息，如名字、地址、长度、属性等。

节区头

代码13-9 IMAGE_SECTION_HEADER结构体

```
#define IMAGE_SIZEOF_SHORT_NAME 8
```

```
typedef struct _IMAGE_SECTION_HEADER {  
    BYTE    Name[IMAGE_SIZEOF_SHORT_NAME];  
    union {  
        DWORD    PhysicalAddress;  
        DWORD    VirtualSize;  
    } Misc;  
    DWORD    VirtualAddress;  
    DWORD    SizeOfRawData;  
    DWORD    PointerToRawData;  
    DWORD    PointerToRelocations;  
    DWORD    PointerToLinenumbers;  
    WORD     NumberOfRelocations;  
    WORD     NumberOfLinenumbers;  
    DWORD    Characteristics;  
} IMAGE_SECTION_HEADER. *PIMAGE_SECTION_HEADER;
```

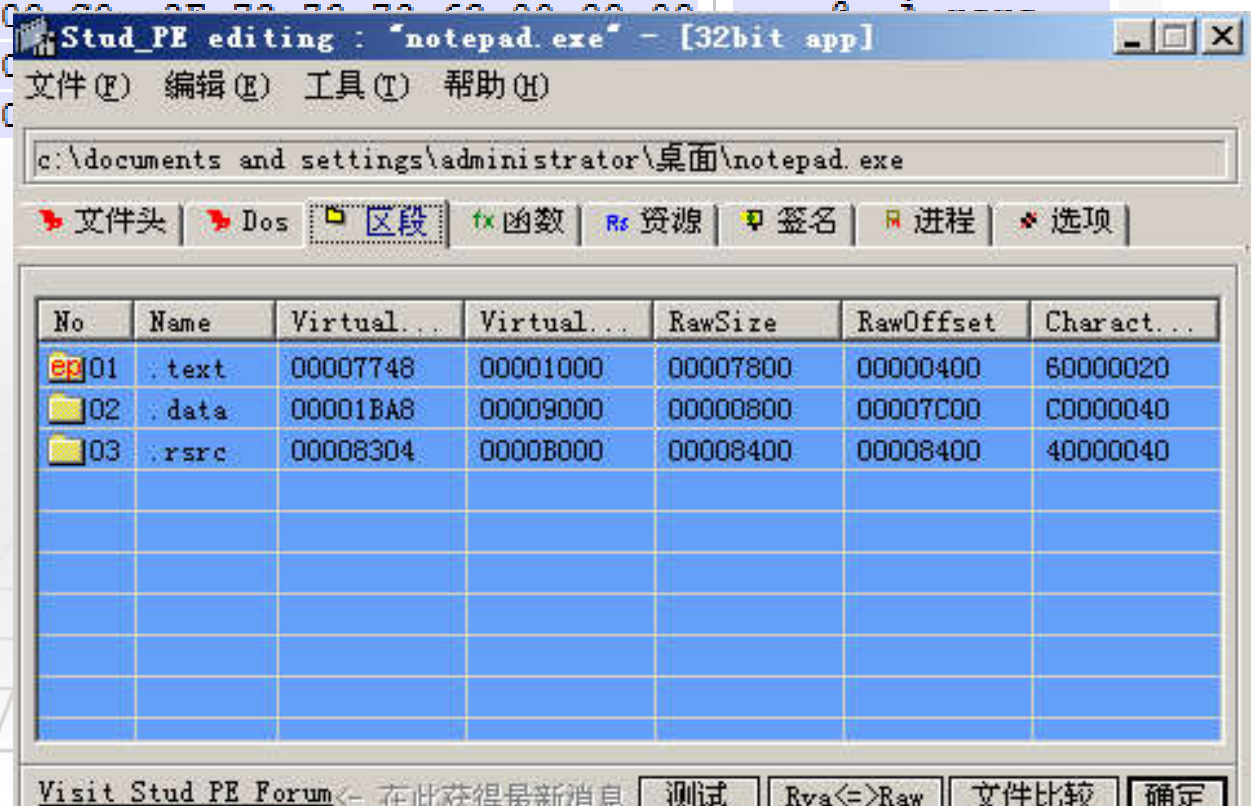
// OBJ文件中表示本节物理地址
// EXE文件中表示节的实际字节数

// 本节的RVA
// 本节在文件中的尺寸
// 本节在文件中的偏移地址
// OBJ文件中表示本节重定位信
// 息的偏移, EXE文件中无意义
// 行号偏移
// 重定位项的数目
// 本节在行号表中的行号数目
// 节属性代码

节区头

- 1、VirtualSize: 内存中节区所占大小
- 2、VirtualAddress: 内存中节区起始地址（RVA）
- 3、SizeOfRawData: 磁盘文件中节区所占大小
- 4、PointerToRawData: 磁盘文件中节区起始位置
- 5、Characteristics: 节区属性

000001D0	00 00 00 00 00 00 00 00 00	2E 74 65 78 74 00 00 00	.text
000001E0	48 77 00 00 00 10 00 00	00 78 00 00 00 04 00 00	Hw x
000001F0	00 00 00 00 00 00 00 00	00 00 00 00 20 00 00 60	
00000200	2E 64 61 74 61 00 00 00	A8 1B 00 00 00 90 00 00	.data
00000210	00 08 00 00 00 7C 00 00	00 00 00 00 00 00 00 00	
00000220	00 00 00 00 40 00 00 00	00 00 00 00 00 00 00 00	
00000230	04 83 00 00 00 B0 00 00	00 00 00 00 00 00 00 00	
00000240	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	



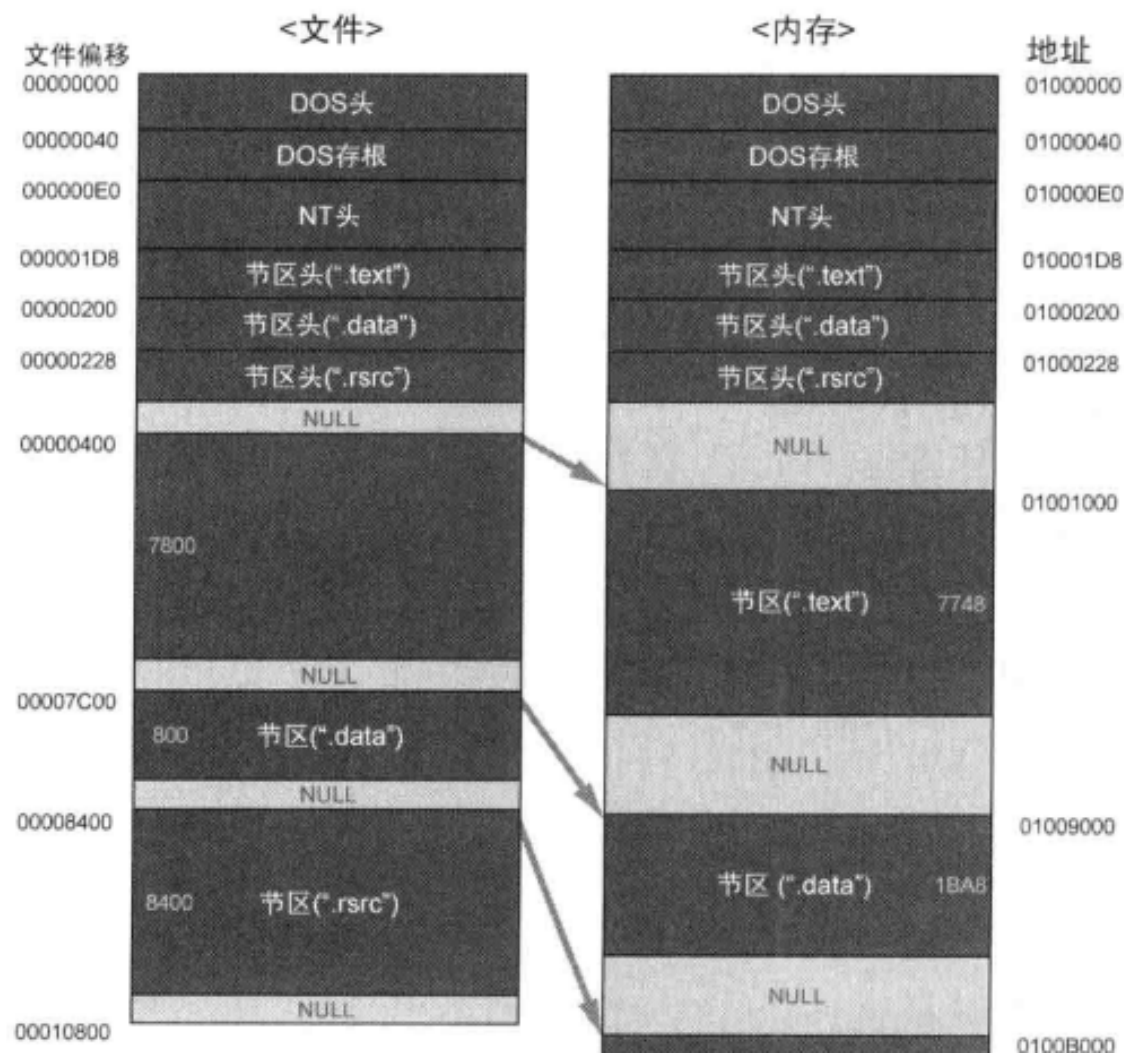
地址的概念--RVA to RAW

- ❑ RVA指内存地址，RAW指文件偏移，
- ❑ $RAW - \text{PointerToRawData} = RVA - \text{VirtualAddress}$
- ❑ $RAW = RVA - \text{VirtualAddress} + \text{PointerToRawData}$
- ❑ 本程序中
- ❑ $\text{PointerToRawData} = 0x00000400$
- ❑ $\text{VirtualAddress} = 0x00001000$

代码13-10 Characteristics

```
#define IMAGE_SCN_CNT_CODE          0x00000020 // Section
                                         contains code.
#define IMAGE_SCN_CNT_INITIALIZED_DATA 0x00000040 // Section contains
                                         initialized data.
#define IMAGE_SCN_CNT_UNINITIALIZED_DATA 0x00000080 // Section contains
                                         uninitialized data.
#define IMAGE_SCN_MEM_EXECUTE       0x20000000 // Section is
                                         executable.
#define IMAGE_SCN_MEM_READ          0x40000000 // Section is
                                         readable.
#define IMAGE_SCN_MEM_WRITE         0x80000000 // Section is
                                         writable.
```


看一下这个图



节

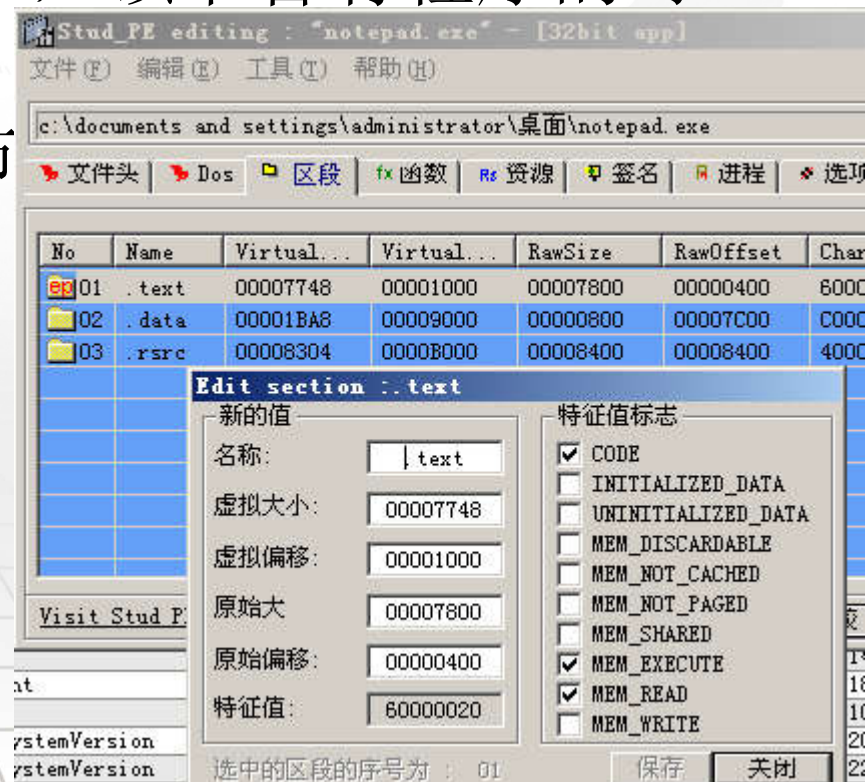
- “节(Section)”跟在节表之后，一般PE文件都有几个“节”。比较常见的有：
- 比较常见的有：
- ✓代码节
- ✓已初始化的数据节
- ✓未初始化的数据节
- ✓引入函数节
- ✓引出函数节
- ✓资源节

代码节

notepad.exe		pFile	Raw Data	Value
IMAGE_DOS_HEADER		00000400	EF 6F F5 77 17 6C F5 77 25 BA F7 77 05 BD F7 77	. o . w . l . w % . . w . . . w
MS-DOS Stub Program		00000410	AB 7A F5 77 42 78 F5 77 57 D7 F5 77 00 00 00 00	. z . w B x . w V . . w
IMAGE_NT_HEADERS		00000420	70 D2 16 77 00 00 00 00 19 DC E3 77 05 4A E5 77	p . . w w . J . w
Signature		00000430	A9 DE E3 77 5F 6E E2 77 56 F4 E3 77 9D 7F E2 77	. . . w _ n . w V . . w . . w
IMAGE_FILE_HEADER		00000440	28 BE E2 77 25 4B E5 77 BB A5 E2 77 AC 7E E2 77	(. . w % K . w . . . w . ~ . w
IMAGE_OPTIONAL_HEADER		00000450	95 56 E5 77 22 FB E4 77 C1 61 E2 77 B3 83 E2 77	. V . w " . . w . a . w . . w
IMAGE_SECTION_HEADER .text		00000460	69 5A E2 77 8F 93 E2 77 FA 6B E2 77 B9 7D E2 77	i Z . w . . w . k . w . } . w
IMAGE_SECTION_HEADER .data		00000470	DB 5E E2 77 B7 D4 E2 77 05 3A E3 77 AE 3A E3 77	. ^ . w . . w . : . w . : . w
IMAGE_SECTION_HEADER .rsrc		00000480	10 94 E2 77 70 5B E2 77 00 00 00 00 B8 97 80 7C	. . . w p [. w
BOUND_IMPORT Directory Table		00000490	2E 93 80 7C B7 A4 80 7C 64 A8 80 7C A0 9F 80 7C	 d
BOUND_IMPORT DLL Names		000004A0	8D 37 83 7C EB 3F 83 7C A9 FF 80 7C 12 FF 80 7C	. 7 . . ?
SECTION .text		000004B0	FD 0C 81 7C 20 94 80 7C E9 17 80 7C 1A 1E 80 7C	
SECTION .data		000004C0	85 DE 80 7C FD 49 84 7C 7B 1D 80 7C 31 B7 80 7C	 { . . 1 . .
SECTION .rsrc		000004D0	F2 1E 80 7C BF FC 80 7C F2 15 81 7C BF 99 80 7C	
		000004E0	1D 9A 80 7C 99 9A 80 7C C9 2E 83 7C EE A3 80 7C	
		000004F0	35 2F 83 7C 56 A6 87 7C 07 9B 80 7C FA BA 80 7C	5 . V

代码节

- 代码节一般名为.text或CODE，该节含有程序的可执行代码。
- 每个PE文件应该都有代码节



吾爱破解 - notepad.exe - [LCG - 主线程, 模块 - notepad]

文件(F) 查看(V) 调试(D) 插件(P) 选项(O) 窗口(W) 帮助(H) [+] 快捷菜单 Tools BreakPoint->

暂停

```
0100739D $ 6A 70 push 0x70
0100739F . 48 98180001 push notepad.01001398
010073A4 . E8 BF010000 call notepad.01007568
010073A9 . 33DB xor ebx,ebx
010073AB . 53 push ebx
010073AC . 8E3D 0C100100 mov edi,dword ptr ds:[<KERNEL32.GetMod
010073B2 . FFD7 call edi
010073B4 . 66:8130 4D5A cmp word ptr ds:[eax],0x5A4D
010073B9 . 75 1F jnz short notepad.010073D0
010073BB . 8B78 3C mov ecx,dword ptr ds:[eax+0x3C]
010073BE . 03C8 add ecx,eax
010073C0 . 8139 50450000 cmp dword ptr ds:[ecx],0x4550
010073C6 . 75 12 jnz short notepad.010073DA
010073C8 . 0FB741 18 movzx eax,word ptr ds:[ecx+0x18]
010073CC . 3D 0B010000 cmp eax,0x10B
010073D1 . 74 1F je short notepad.010073F2
010073D3 . 3D 0B020000 cmp eax,0x20B
010073D8 . 74 05 je short notepad.010073DF
010073DA > 895D E4 mov dword ptr ss:[ebp-0x1C],ebx
010073DD . EB 27 jmp short notepad.01007406
010073DF > 83B9 84000000 cmp dword ptr ds:[ecx+0x84],0xE
010073E6 . 76 F2 jbe short notepad.010073DA
010073E8 . 33C0 xor eax,eax
010073EA . 3999 F8000000 cmp dword ptr ds:[ecx+0xF8],ebx
010073F0 . EB 0E jmp short notepad.01007400
010073F2 > 8379 74 0E cmp dword ptr ds:[ecx+0x74],0xE
010073F6 . 76 E2 jbe short notepad.010073DA
010073F8 . 33C0 xor eax,eax
010073FA . 3999 E8000000 cmp dword ptr ds:[ecx+0xE8],ebx
```

pMod
kern
GetM

Stud PE editing : "notepad.exe" - [32bit app]

文件(F) 编辑(E) 工具(T) 帮助(H)

c:\documents and settings\administrator\桌面\notepad.exe

文件头 | Dos | 区段 | fx函数 | 资源 | 签名 | 进程 | 选项

No	Name	Virtual...	Virtual...	RawSize	RawOffset	Charac
01	.text	00007748	00001000	00007800	00000400	600000
02	.data	00001BA8	00009000	00000800	00007C00	C00000
03	.rsrc	00008304	0000B000	00008400	00008400	400000

Edit section : .text

新的值

名称: | text

虚拟大小: 00007748

虚拟偏移: 00001000

原始大: 00007800

原始偏移: 00000400

特征值: 60000020

特征值标志

- ☒ CODE
- ☐ INITIALIZED_DATA
- ☐ UNINITIALIZED_DATA
- ☐ MEM_DISCARDABLE
- ☐ MEM_NOT_CACHED
- ☐ MEM_NOT_PAGED
- ☐ MEM_SHARED
- ☒ MEM_EXECUTE
- ☒ MEM_READ
- ☐ MEM_WRITE

选中的区段的序号为: 01

保存 关闭

数据节

0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF

7BF0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7C00h: 00 00 00 00 D4 70 00 01 00 00 00 00 00 00 00 00
7C10h: 00 00 00 00 00 00 00 00 78 00 00 00 01 00 00 00
7C20h: 4E 00 6F 00 74 00 65 00 70 00 61 00 64 00 00 00
7C30h: FF FF FF FF 01 00 00 00 02 00 00 00 03 00 00 00
7C40h: 04 00 00 00 05 00 00 00 06 00 00 00 07 00 00 00
7C50h: 08 00 00 00 09 00 00 00 0A 00 00 00 0B 00 00 00
7C60h: 0C 00 00 00 0D 00 00 00 0E 00 00 00 0F 00 00 00
7C70h: 10 00 00 00 11 00 00 00 12 00 00 00 13 00 00 00
7C80h: 2D 00 00 00 14 00 00 00 15 00 00 00 16 00 00 00
7C90h: 17 00 00 00 18 00 00 00 19 00 00 00 1A 00 00 00
7CA0h: 1B 00 00 00 1C 00 00 00 1D 00 00 00 1E 00 00 00
7CB0h: 1F 00 00 00 20 00 00 00 21 00 00 00 22 00 00 00
7CC0h: 23 00 00 00 24 00 00 00 25 00 00 00 26 00 00 00
7CD0h: 27 00 00 00 28 00 00 00 29 00 00 00 2A 00 00 00
7CE0h: 2B 00 00 00 2C 00 00 00 34 90 00 01 38 90 00 01

.....
....Op.....
.....x.....
N.o.t.e.p.a.d...
yyyy.....

Stud_PE editing : "notepad.exe" - [32bit app]
文件(F) 编辑(E) 工具(T) 帮助(H)
c:\documents and settings\administrator\桌面\notepad.exe
文件头 Dos 区段 fx函数 Rs资源 签名 进程 选项

No	Name	Virtual...	Virtual...	RawSize	RawOffset	Charact...
01	.text	00007748	00001000	00007800	00000400	60000020
02	.data	00001BA8	00009000	00000800	00007C00	C0000040
03	.rsrc	00008304	0000B000	00008400	00008400	40000040

Visit Stud PE Forum < 在此获

新的值
名称: .data
虚拟大小: 00001BA8
虚拟偏移: 00009000
原始大: 00000800
原始偏移: 00007C00
特征值: C0000040

特征值标志
☐ CODE
☒ INITIALIZED_DATA
☐ UNINITIALIZED_DATA
☐ MEM_DISCARDABLE
☐ MEM_NOT_CACHED
☐ MEM_NOT_PAGED
☐ MEM_SHARED
☐ MEM_EXECUTE
☒ MEM_READ
☒ MEM_WRITE

Template Results - PE.bt

Name
struct IMAGE_DOS_HEADER DosHeader
struct IMAGE_DOS_STUB DosStub
struct IMAGE_NT_HEADERS NtHeader
DWORD Signature
struct IMAGE_FILE_HEADER FileHeader
struct IMAGE_OPTIONAL_HEADER32 OptionalHeader
struct IMAGE_SECTION_HEADER SectionHeaders[3]
struct IMAGE_SECTION_DATA Section[0]
struct IMAGE_SECTION_DATA Section[1]
struct IMAGE_SECTION_DATA Section[2]
struct IMAGE_IMPORT_DESCRIPTOR ImportDescriptor[0]

.text	400h
.data	7C00h
.rsrc	8400h
comdlg32.dll	8A04h

未初始的数据节

- ▶ 这个节的名称一般叫.bbs。
- ▶ 这个节里放有未初始化的全局变量和静态变量。
 - 例如"static int k;"
- ▶ 该节的'PointerToRawData'为0，表明其内容不在文件内，特征位'IMAGE_SCN_CNT_UNINITIALIZED_DATA'指明所有内容必须在加载时间置0。这意味着有节头，但没有节在文件内，该节被加载器创建，并且包含全0字节。其长度是'SizeOfUninitializedData'。典型名字如'.bss', 'BSS'。

回顾 第一个逆向的例子

- 调用了MESSAGEBOX,
- 运行时, 需要加载动态链接库user32.dll, 填充地址。。。

IAT

- IAT即Import Address Table，**导入地址表**，是一种表格，用来记录程序正在使用哪些库中哪些函数。（Table即指数组）
- IMAGE_IMPORT_DESCRIPTOR对应每一个被导入的DLL
- 一系列DWORD的数组。
- 在文件中时，每个双字中存放着对应引入函数的函数名字符串的RVA
- 在内存中时，每个双字中存放着对应引入函数的地址。
- 转载函数实现装载。。。。

IAT

PEview - C:\Documents and Settings\Administrator\桌面\notepad.exe

文件(F) 视图(V) 前往(G) 帮助(H)

notepad.exe

- IMAGE_DOS_HEADER
- MS-DOS Stub Program
- IMAGE_NT_HEADERS
 - IMAGE_SECTION_HEADER .text
 - IMAGE_SECTION_HEADER .data
 - IMAGE_SECTION_HEADER .rsrc
 - BOUND_IMPORT Directory Table
 - BOUND_IMPORT DLL Names
- SECTION .text
 - IMPORT Address Table**
 - IMAGE_DEBUG_DIRECTORY
 - IMAGE_LOAD_CONFIG_DIRECTORY
 - IMAGE_DEBUG_TYPE_CODEVIEW
 - IMPORT Directory Table
 - IMPORT Name Table
 - IMPORT Hints/Names & DLL Names
- SECTION .data
- SECTION .rsrc

pFile	Data	Description	Value
00000400	77F56FEF	Virtual Address	01EF RegQueryValueExW
00000404	77F56C17	Virtual Address	01CA RegCloseKey
00000408	77F7BA25	Virtual Address	01D0 RegCreateKeyW
0000040C	77F7BD05	Virtual Address	0139 IsTextUnicode
00000410	77F57AAB	Virtual Address	01EE RegQueryValueExA
00000414	77F57842	Virtual Address	01E4 RegOpenKeyExA
00000418	77F5D757	Virtual Address	01FC RegSetValueExW
0000041C	00000000	End of Imports	ADVAPI32.dll
00000420	7716D270	Virtual Address	0008 CreateStatusWindowW
00000424	00000000	End of Imports	COMCTL32.dll
00000428	77E3DC19	Virtual Address	0098 EndPage
0000042C	77E54A05	Virtual Address	0000 AbortDoc
00000430	77E3DEA9	Virtual Address	0096 EndDoc
00000434	77E26E5F	Virtual Address	008C DeleteDC
00000438	77E3F456	Virtual Address	0249 StartPage
0000043C	77E27F9D	Virtual Address	01B6 GetTextExtentPoint32W
00000440	77E2BE28	Virtual Address	002F CreateDCW
00000444	77E54B25	Virtual Address	0211 SetAbortProc
00000448	77F2A5RR	Virtual Address	01RC GetTextFaceW

Import Directory Table

输入目录是一个IMAGE_IMPORT_DESCRIPTORs数组，每个DLL都使用一个元素，该列表中止于一个全0的数组元素。

一个IMAGE_IMPORT_DESCRIPTOR是一个具有如下元素的结构：

```
IMAGE_IMPORT_DESCRIPTOR STRUCT
    union
        Characteristics dd    ?
        OriginalFirstThunk dd ? //指向引入函数名列表或序号列表 INT
    Ends
    TimeDateStamp dd    ?
    ForwarderChain dd    ?
    Name1 dd            ?      //指向dll函数名
    FirstThunk dd        ?      //指向IAT表
IMAGE_IMPORT_DESCRIPTOR ENDS
```

输入目录是一个IMAGE_IMPORT_DESCRIPTORs数组，每个DLL都使用一个元素，该列表中止于一个全0的数组元素。

一个IMAGE_IMPORT_DESCRIPTOR是一个具有如下元素的结构：

IMAGE_IMPORT_DESCRIPTOR STRUCT

union

Characteristics dd ?

OriginalFirstThunk dd ? //指向引入函数名列表或序号列表

Ends

TimeStamp dd ?

ForwarderChain dd ?

Name1 dd ? //指向dll函数名

FirstThunk dd ? //指向IAT表

IMAGE_IMPORT_DESCRIPTOR ENDS

69C0h:	CC CC CC CC	CC CC FF 25	EC 12 00 01	CC CC CC CC
69D0h:	CC CC FF 25	24 13 00 01	CC CC CC CC	CC 68 00 00
69E0h:	03 00 68 00	00 01 00 E8	10 00 00 00	59 59 C3 CC
69F0h:	CC CC CC CC	33 CD C3 CC	CC CC CC CC	FF 25 3C 13
6A00h:	00 01 CC CC	90 79 00 00	FF FF FF FF	FF FF FF FF
6A10h:	AC 7A 00 00	C4 12 00 00	40 78 00 00	FF FF FF FF
6A20h:	FF FF FF FF	FA 7A 00 00	74 11 00 00	80 79 00 00
6A30h:	FF FF FF FF	FF FF FF FF	3A 7B 00 00	B4 12 00 00
6A40h:	EC 76 00 00	FF FF FF FF	FF FF FF FF	5E 7B 00 00
6A50h:	CC 40 00 00	80 79 00 00	FF FF FF FF	FF FF FF FF

Template Results - PE.bt

Name	
...DWRN SizeOfHeapCommit	10000

输入目录是一个IMAGE_IMPORT_DESCRIPTORs数组，每个DLL都使用一个元素，该列表中止于一个全0的数组元素。

一个IMAGE_IMPORT_DESCRIPTOR是一个具有如下元素的结构：

```
IMAGE_IMPORT_DESCRIPTOR STRUCT
union
    Characteristics dd    ?
    OriginalFirstThunk dd ? //指向引入函数名列表或序号列表
Ends
TimeStamp dd    ?
ForwarderChain dd ?
Name1 dd        ?          //指向dll函数名
FirstThunk dd    ?          //指向IAT表
IMAGE_IMPORT_DESCRIPTOR ENDS
```

我们着重关心两个指针，OriginalFirstThunk和FirstThunk。

Characteristics一词出于历史原因，故在此给它了一个别名，OriginalFirstThunk。

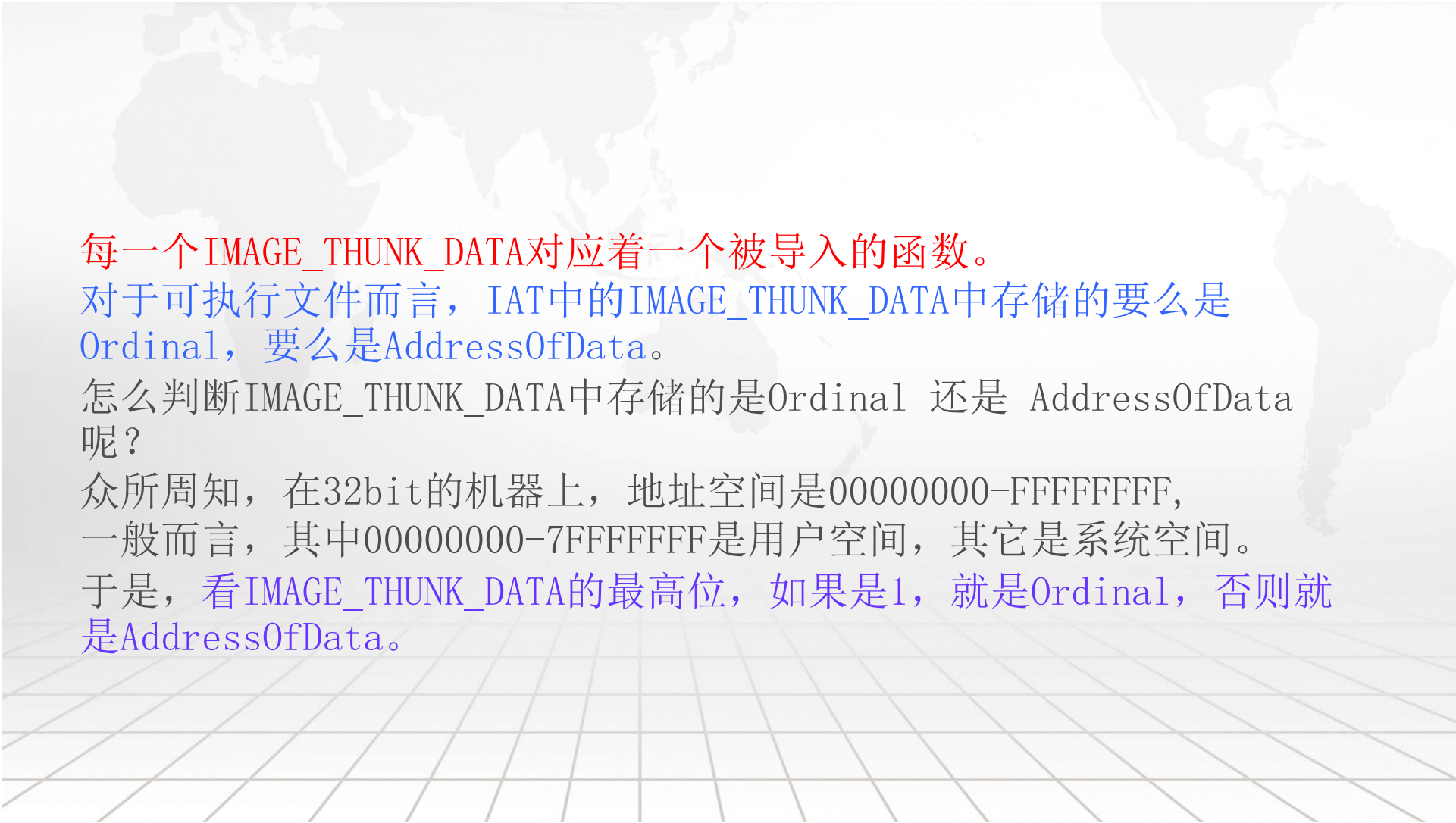
OriginalFirstThunk和FirstThunk是两个DWORD值，存贮着两个RVA数值，其实它们就是两个指针。

OriginalFirstThunk和FirstThunk实际上都是指向同一个数组。

前者，我们称之为INT，而后者，我们称之为IAT。

IAT是一个IMAGE_THUNK_DATA类型的数组。有多少个函数被导入，这个数组就有多少个成员。该数组以0结尾。

```
typedef struct _IMAGE_THUNK_DATA32 {  
    union {  
        DWORD ForwarderString;    // 一个RVA地址，指向forwarder string  
        DWORD Function;          // PDWORD，被导入的函数的入口地址  
        DWORD Ordinal;           // 该函数的序数  
        DWORD AddressOfData;     // 一个RVA地址，指向IMAGE_IMPORT_BY_NAME  
    } u1;  
} IMAGE_THUNK_DATA32;
```

每一个IMAGE_THUNK_DATA对应着一个被导入的函数。

对于可执行文件而言，IAT中的IMAGE_THUNK_DATA中存储的要么是Ordinal，要么是AddressOfData。

怎么判断IMAGE_THUNK_DATA中存储的是Ordinal 还是 AddressOfData 呢？

众所周知，在32bit的机器上，地址空间是00000000-FFFFFFFF，一般而言，其中00000000-7FFFFFFF是用户空间，其它是系统空间。

于是，看IMAGE_THUNK_DATA的最高位，如果是1，就是Ordinal，否则就是AddressOfData。

PEview - C:\Documents and Settings\Administrator\桌面\notepad.exe

文件(F) 视图(V) 前往(G) 帮助(H)

BP P VB Notepad Calc Folder CMD

notepad.exe

- IMAGE_DOS_HEADER
- MS-DOS Stub Program
- IMAGE_NT_HEADERS
 - Signature
 - IMAGE_FILE_HEADER
 - IMAGE_OPTIONAL_HEADER
 - IMAGE_SECTION_HEADER .text
 - IMAGE_SECTION_HEADER .data
 - IMAGE_SECTION_HEADER .rsrc
 - BOUND_IMPORT Directory Table
 - BOUND_IMPORT DLL Names
- SECTION .text
 - IMPORT Address Table
 - IMAGE_DEBUG_DIRECTORY
 - IMAGE_LOAD_CONFIG_DIRECTORY
 - IMAGE_DEBUG_TYPE_CODEVIEW
 - IMPORT Directory Table
 - IMPORT Name Table**
 - IMPORT Hints/Names & DLL Names
- SECTION .data
- SECTION .rsrc

pFile	Data	Description	Value
00006D8C	00000000	End of Imports	WINSPOOL.DRV
00006D90	00007A7A	Hint/Name RVA	000F PageSetupDlgW
00006D94	00007A5E	Hint/Name RVA	0006 FindTextW
00006D98	00007A9E	Hint/Name RVA	0012 PrintDlgExW
00006D9C	00007A50	Hint/Name RVA	0003 ChooseFontW
00006DA0	00007A40	Hint/Name RVA	0008 GetFileNameW
00006DA4	00007A8A	Hint/Name RVA	000A GetOpenFileNameW
00006DA8	00007A6A	Hint/Name RVA	0015 ReplaceTextW
00006DAC	00007A14	Hint/Name RVA	0004 CommDlgExtendedError
00006DB0	00007A2C	Hint/Name RVA	000C GetSaveFileNameW
00006DB4	00000000	End of Imports	comdlg32.dll
00006DB8	00007BDC	Hint/Name RVA	004E _XcptFilter
00006DBC	00007BD4	Hint/Name RVA	00F6 _exit
00006DC0	00007BCA	Hint/Name RVA	00C5 _c_exit
00006DC4	00007BC2	Hint/Name RVA	0317 time
00006DC8	00007BB6	Hint/Name RVA	02D4 localtime
00006DCC	00007BEA	Hint/Name RVA	00C8 _cexit
00006DD0	00007BA0	Hint/Name RVA	02C6 iswctype
00006DD4	00007B8C	Hint/Name RVA	00ED _except_handler3
00006DD8	00007B84	Hint/Name RVA	0274 _wtol
00006DDC	00007B7A	Hint/Name RVA	032F wcsncmp
00006DE0	00007B6C	Hint/Name	
00006DE4	00007BF4	Hint/Name	
00006DE8	00007BFC	Hint/Name	
00006DEC	00007C06	Hint/Name	

7990H-1000H+400H

Data最高位:
 为1时, 表示通过序号引入函数
 为0时, 表示通过函数名引入

Startup notepad.exe																	
Edit As: Hex Run Script Run Template: PE.bt																	
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0123456789ABCDEF
6E30h:	74	53	61	76	65	46	69	6C	65	4E	61	6D	65	57	00	00	tSaveFileNameW..
6E40h:	08	00	47	65	74	46	69	6C	65	54	69	74	6C	65	57	00	..GetFileTitleW.
6E50h:	03	00	43	68	6F	6F	73	65	46	6F	6E	74	57	00	06	00	..ChooseFontW...
6E60h:	46	69	6E	64	54	65	78	74	57	00	15	00	52	65	70	6C	FindTextW...Repl
6E70h:	61	63	65	54	65	78	74	57	00	00	0F	00	50	61	67	65	aceTextW...Page
6E80h:	53	65	74	75	70	44	6C	67	57	00	0A	00	47	65	74	4F	SetupDlgW...GetO
6E90h:	70	65	6E	46	69	6C	65	4E	61	6D	65	57	00	00	12	00	penFileNameW....
6EA0h:	50	72	69	6E	74	44	6C	67	45	78	57	00	63	6F	6D	64	PrintDlgExW.comd
6EB0h:	6C	67	33	32	2E	64	6C	6C	00	00	03	01	53	68	65	6C	lg32.dll....Shel
Template Results - PE.bt																	

图13-11描述了notepad.exe之kernel32.dll的IMAGE_IMPORT_DESCRIPTOR结构。

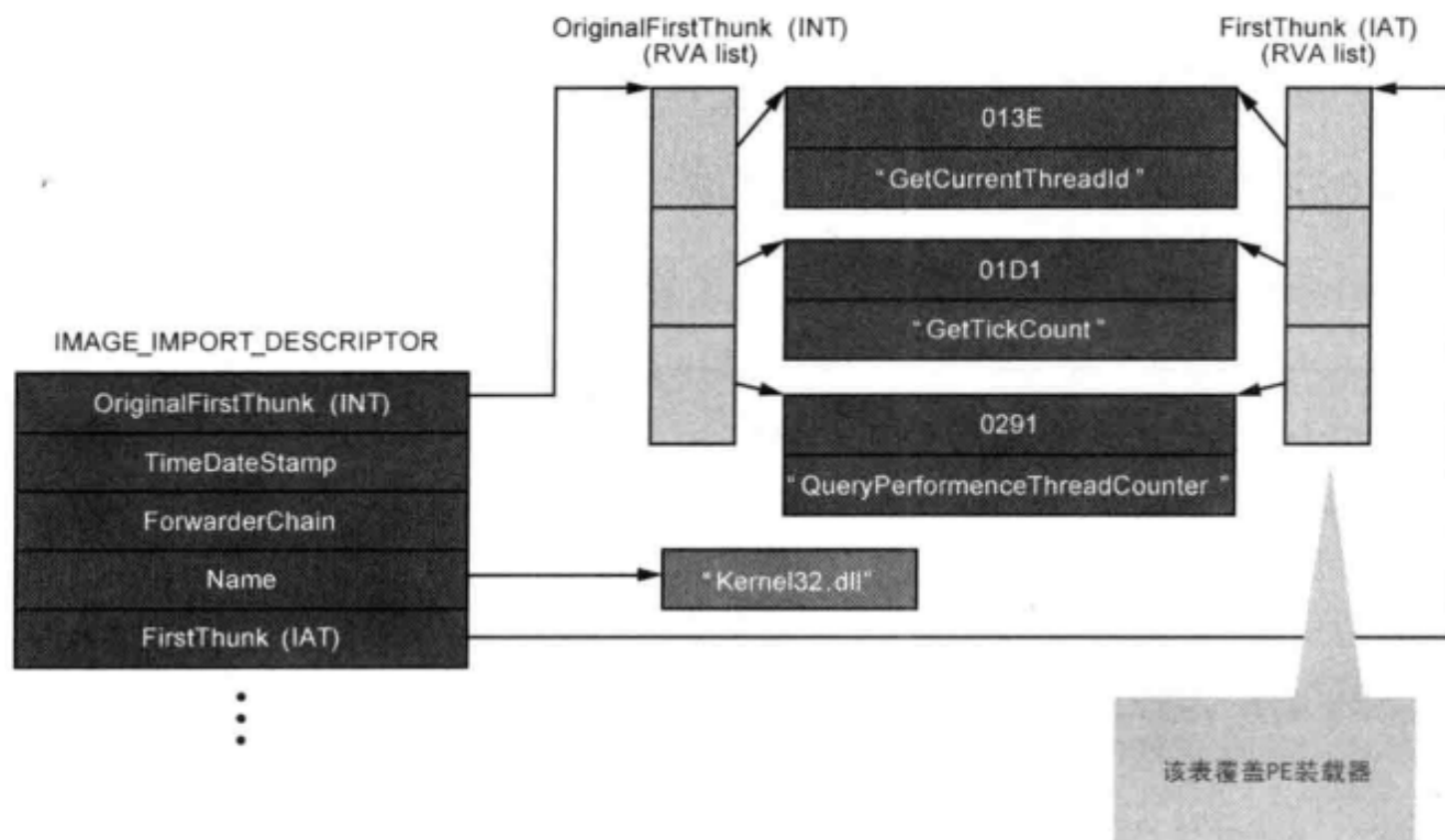


图13-11 IAT

PE装载器把导入函数输入至IAT的顺序：

1. 读取IID的Name成员，获取**库名称**字符串（kernel32.dll）
2. 装载相应库：LoadLibrary("kernel32.dll")
3. 读取IID的OriginalFirstThunk成员，**获取INT地址**
4. 逐一读取INT中数组的值，**获取相应的IMAGE_IMPORT_BY_NAME地址（RVA）**
5. 使用IMAGE_IMPORT_BY_NAME的Hint（ordinary：库中函数的固有编号，2字节）或Name项，**获取相应函数的起始地址：GetProcAddress("GetCurrentThreadId")**
6. 读取IID的FirstNameThunk(IAT)成员，**获得IAT地址**
7. 将**上面获得的函数地址**输入到相应的IAT数组中。
8. 重复以上步骤4~7，直到INT结束（遇到NULL时）。

注意上述过程中获得的地址均为RVA地址，需要将其转换为RWA地址后才可在文件中查找。

15.3节

RVA of Import Directory

Workspace

- Open Files
 - C:\debuger\...\Data\PE.bt
 - C:\...\桌面\notepad.exe (Read Only)
- Favorite Files
- Recent Files
- Bookmarked Files

Inspector

Type	Value
Signed Byte	4
Unsigned Byte	4
Signed Short	30212
Unsigned Short	30212
Signed Int	30212
Unsigned Int	30212
Signed Int64	858993489412
Unsigned Int64	858993489412
Float	4.233603e-41
Double	4.24399173119766e-312
Half Float	24640
String	v
Unicode	楷

Startup notepad.exe

Edit As: Hex Run Script Run Template: PE.bt

Offset	Hex	ASCII
00E0h	50 45 00 00 4C 01 03 00	PE
00F0h	00 00 00 00 E0 00 0F 01	
0100h	00 8C 00 00 00 00 00 00	
0110h	00 90 00 00 00 00 00 01	
0120h	05 00 01 00 05 00 01 00	
0130h	00 40 01 00 00 04 00 00	
0140h	00 00 04 00 10 01 00 00	
0150h	00 00 00 00 10 00 00 00	
0160h	04 76 00 00 C8 00 00 00	
0170h	00 00 00 00 00 00 00 00	
0180h	00 00 00 00 00 00 00 50	
0190h	00 00 00 00 00 00 00 00	
01A0h	00 00 00 00 00 00 00 A8	
01B0h	50 02 00 00 D0 00 00 00	
01C0h	00 00 00 00 00 00 00 00	
01D0h	00 00 00 00 00 00 00 2E	
01E0h	48 77 00 00 10 00 00 00	

Template Results - PE.bt

Name	Value	Start	Size	Color
struct IMAGE_DATA_DIRECTORY_ARRAY DataDirArray		158h	80h	Fg: Bg:
struct IMAGE_DATA_DIRECTORY Export		158h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY Import		160h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY Resource		168h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY Exception		170h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY Security		178h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY BaseRelocationTable		180h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY DebugDirectory		188h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY CopyrightOrArchitectureSpecificData		190h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY GlobalPtr		198h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY TLSDirectory		1A0h	8h	Fg: Bg: IMAGE_DI
struct IMAGE_DATA_DIRECTORY LoadConfigurationDirectory		1A8h	8h	Fg: Bg: IMAGE_DI

表13-7 notepad.exe文件的第一个IMAGE_IMPORT_DESCRIPTOR结构体

文件偏移	成 员	RVA	RAW
6A04	OriginalFirstThunk(INT)	00007990	00006D90
6A08	TimeStamp	FFFFFFFF	-
6A0C	ForwarderChain	FFFFFFFF	-
6A10	Name	00007AAC	00006EAC
6A14	FirstThunk(IAT)	000012C4	000006C4

1、库名称（Name）

- 在文件中6EAC可得（RVA:7AAC -> RAW:6EAC）库名称cmdlg32.dll,后面两个00

00006EA0	50 72 69 6E 74 44 6C 67	45 78 57 00 63 6F 6D 64	PrintDlgExW	cmd
00006EB0	6C 67 33 32 2E 64 6C 6C	00 00 03 01 53 68 65 6C	lg32.dll	Shel

2、OriginalFirstThunk-INT

- RVA: 7990 -> RAW: 6D90

00006D90	7A 7A 00 00 5E 7A 00 00 9E 7A 00 00 50 7A 00 00	zz ^z žz E
00006DA0	40 7A 00 00 8A 7A 00 00 6A 7A 00 00 14 7A 00 00	@z Šz jz
00006DB0	2C 7A 00 00 00 00 00 00 DC 7B 00 00 D4 7B 00 00	,z ů{ Ć

3. IMAGE_IMPORT_BY_NAME

RVA: 7A7A即为RAW: 6E7A。

文件偏移6E7A最初的2个字节值(000F)为Ordinal, 是库中函数的固有编号。Ordinal的后面为函数名称字符串PageSetupDlgW(同C语言一样, 字符串末尾以Terminating NULL['\0']结束)。

如图13-16所示, INT是IMAGE_IMPORT_BY_NAME结构体指针数组(参考代码13-12)。数组的第一个元素指向函数的Ordinal值000F, 函数的名称为PageSetupDlgW。

00006E60	46 69 6E 64 54 65 78 74 57 00 15 00 52 65 70 6C	FindTextW...Repl
00006E70	61 63 65 54 65 78 74 57 00 00 0F 00 50 61 67 65	aceTextW...Page
00006E80	53 65 74 75 70 44 6C 67 57 00 0A 00 47 65 74 4F	SetupDlgW...GetO
00006E90	70 65 6E 46 69 6C 65 4E 61 6D 65 57 00 00 12 00	penFileNameW....

图13-16 IMAGE_IMPORT_BY_NAME

4. FirstThunk - IAT (Import Address Table)

IAT的RVA:12C4即为RAW: 6C4。

```
typedef struct _IMAGE_IMPORT_BY_NAME {  
    WORD    Hint;           // 1) 需导入的函数序号  
    BYTE    Name[1];       // 2) 需导入的函数名称  
} IMAGE_IMPORT_BY_NAME, *PIMAGE_IMPORT_BY_NAME;
```

000006B0	00 00 00 00 3C 64 F5 72 40 4D F5 72 91 50 F5 72	<dör@Mör Pör
000006C0	00 00 00 00 06 49 32 76 CE 85 31 76 84 9D 32 76	12v 1v .2v
000006D0	E1 C3 31 76 06 23 30 76 9D 7B 31 76 02 86 31 76	âA1v.#0v.<1v.†1v
000006E0	36 00 31 76 2B 7C 31 76 00 00 00 00 AE 2D 40 4D	6.1v+ 1v....@M
000006F0	9A 9E 40 4D CE 9E 40 4D CF AE 41 4D 69 AB 41 4D	ŠZ@M ž@M @Mi«AM

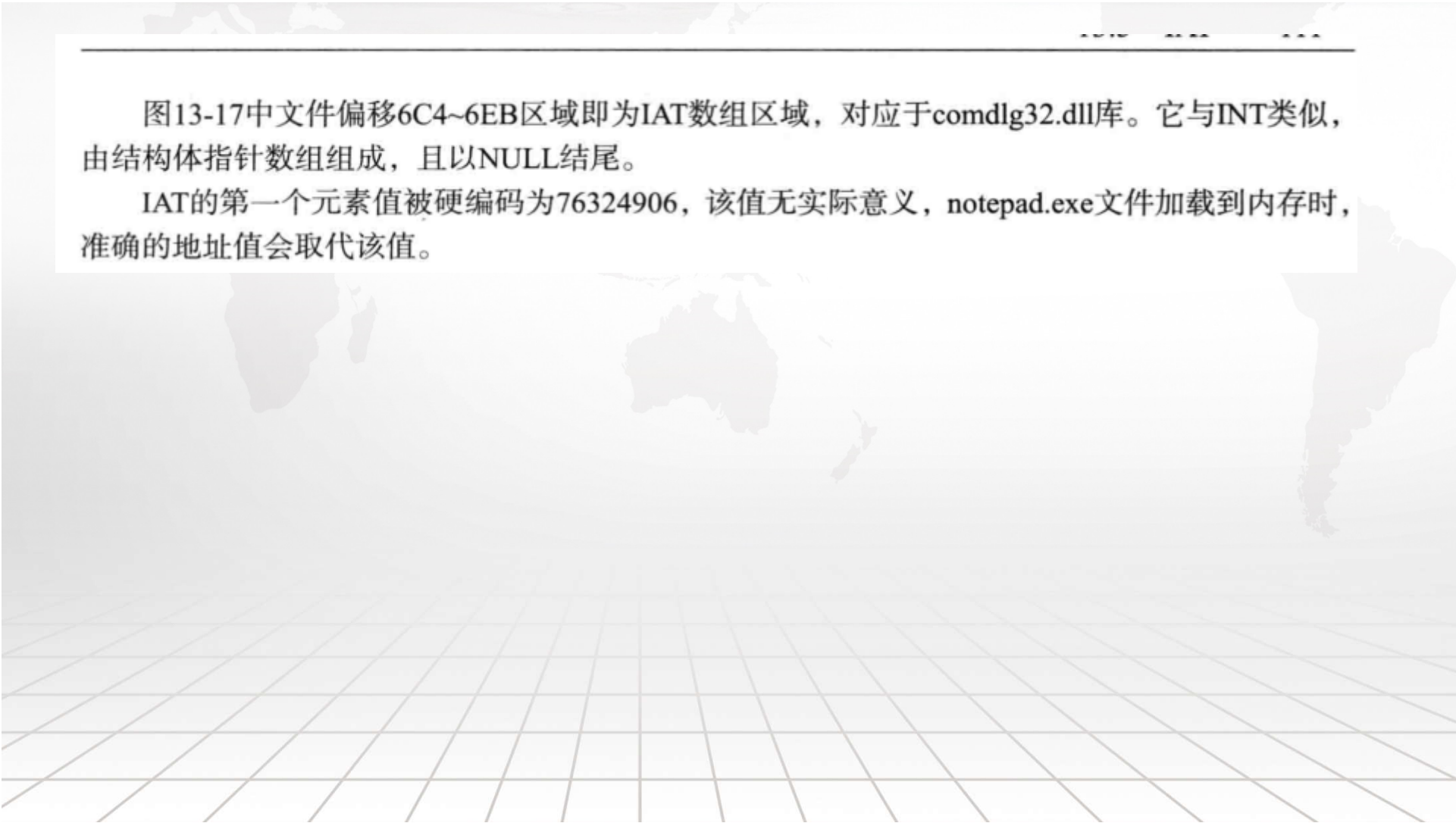


图13-17中文件偏移6C4~6EB区域即为IAT数组区域，对应于comdlg32.dll库。它与INT类似，由结构体指针数组组成，且以NULL结尾。

IAT的第一个元素值被硬编码为76324906，该值无实际意义，notepad.exe文件加载到内存时，准确的地址值会取代该值。

EAT

- EAT是一种核心机制，它使不同的应用程序可以调用库文件中提供的函数，PE文件中的特定结构体（IMAGE_EXPORT_DIRECTORY）保存着导出信息，且PE文件中仅有一个用来说明EAT的IMAGE_EXPORT_DIRECTORY结构体

00000160	00 00 00 00 10 00 00 00	2C 26 00 00 FD 6C 00 00	, & ý1
----------	-------------------------	-------------------------	--------

- 如图为kernel32.dll文件的
IMAGE_OPTIONAL_HEADER32.DataDirectory[0]，可得RVA=262C，所以RAW为1A2C

EAT

代码13-14 IMAGE_EXPORT_DIRECTORY结构体

```
typedef struct _IMAGE_EXPORT_DIRECTORY {
    DWORD    Characteristics;
    DWORD    TimeDateStamp;           // creation time date stamp
    WORD     MajorVersion;
    WORD     MinorVersion;
    DWORD    Name;                    // address of library file name
    DWORD    Base;                    // ordinal base
    DWORD    NumberOfFunctions;       // number of functions
    DWORD    NumberOfNames;           // number of names
    DWORD    AddressOfFunctions;      // address of function start address
                                        array
    DWORD    AddressOfNames;          // address of function name string array
    DWORD    AddressOfNameOrdinals;   // address of ordinal array
} IMAGE_EXPORT_DIRECTORY, *PIMAGE_EXPORT_DIRECTORY;  出处: Microsoft P
```

EAT

代码13-14 IMAGE_EXPORT_DIRECTORY结构体

```
typedef struct _IMAGE_EXPORT_DIRECTORY {
    DWORD Characteristics;
    DWORD TimeDateStamp;           // creation time
    WORD MajorVersion;
    WORD MinorVersion;
    DWORD Name;                    // address of lib
    DWORD Base;                    // ordinal base
    DWORD NumberOfFunctions;       // number of func
    DWORD NumberOfNames;           // number of name
    DWORD AddressOfFunctions;      // address of fun
                                   // array
    DWORD AddressOfNames;          // address of fun
    DWORD AddressOfNameOrdinals;   // address of ord
} IMAGE_EXPORT_DIRECTORY, *PIMAGE_EXPORT_DIRECTORY;
```

- 重要成员：
- 1、NumberOfFunctions：实际Export函数的个数
- 2、NumberOfNames：Export函数中具名的函数个数
- 3、AddressOfFunctions：Export函数地址数组（数组元素个数=NumberOfFunctions）
- 4、AddressOfNames：函数名称地址数组（数组元素个数=NumberOfNames）
- 5、AddressOfNameOrdinals：Ordinal地址数组（数组元素个数=NumberOfNames）

00001A20	8D 45 BC 50 FF 15 44 12 80 7C C3 90	00 00 00 00
00001A30	E1 5B 02 48 00 00 00 00 8E 4B 00 00	01 00 00 00
00001A40	B9 03 00 00 B9 03 00 00 54 26 00 00	38 35 00 00
00001A50	1C 44 00 00 D4 A6 00 00 05 55 03 00	D9 26 03 00

E4Pÿ D € Ä
á [H ŽK
¹ ¹ T& 85
D Œ U Û&

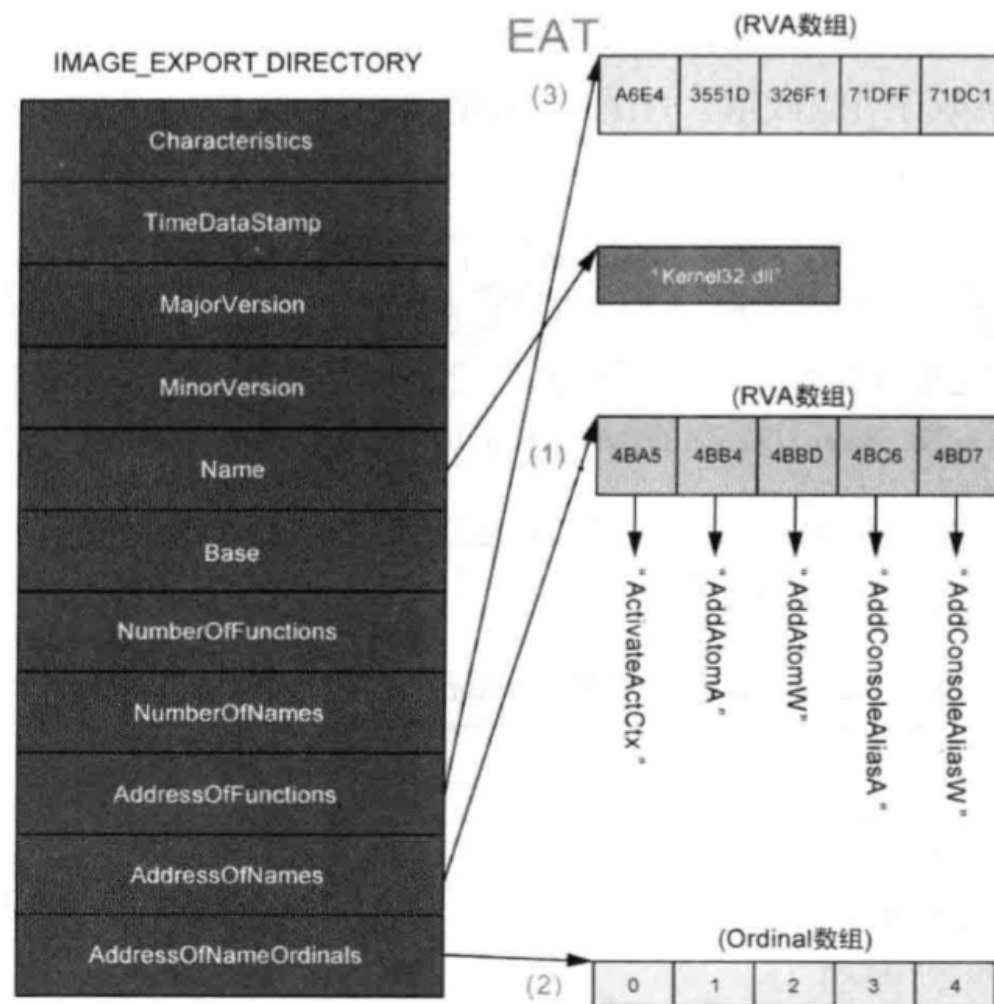


图13-21 EAT

https://blog.csdn.net/diamond_biu

- 从库中获得函数地址的API为GetProcAddress()函数，该API引用EAT来获取指定API的地址。
- 其操作原理如下：利用AddressOfNames成员转到“函数名称数组”“函数名称数组”中存储着字符串地址。通过比较字符串，查找指定的函数名称，获取对应数组的索引name_index
- 利用AddressOfNameOrdinals成员，转到Ordinary数组在ordinary数组中通过name_index查找相应ordinary利用AddressOfFunctions成员转到“函数地址数组”（EAT）在“函数地址数组”中将刚刚求得的ordinal用作数组索引，获得指定函数的起始地址

在kernel32.DLL中

1、函数名称数组

- AddressOfNames成员的值RVA = 3538，即RAW = 2938，

00002930	46 BE 00 00 99 9A 00 00	9B 4B 00 00 AA 4B 00 00	F% ²š >K ²K
00002940	B3 4B 00 00 BC 4B 00 00	CD 4B 00 00 DE 4B 00 00	³K ¼K ÍK ßK
00002950	FD 4B 00 00 1C 4C 00 00	29 4C 00 00 45 4C 00 00	ýK L)L EL
00002960	52 4C 00 00 6C 4C 00 00	7C 4C 00 00 95 4C 00 00	RL 1L L •L
00002970	A3 4C 00 00 AE 4C 00 00	B9 4C 00 00 C5 4C 00 00	£L ©L ²L ÅL
00002980	DD 4C 00 00 F7 4C 00 00	18 4D 00 00 2F 4D 00 00	ÝL ÷L M /M
00002990	47 4D 00 00 5E 4D 00 00	7C 4D 00 00 96 4D 00 00	GM ^M M -M
000029A0	AA 4D 00 00 C3 4D 00 00	E2 4D 00 00 E7 4D 00 00	²M ĂM âM çM
000029B0	FC 4D 00 00 11 4E 00 00	2A 4E 00 00 38 4E 00 00	üM N *N 8N
000029C0	51 4E 00 00 6A 4E 00 00	78 4E 00 00 87 4E 00 00	QN jN xN ‡N
000029D0	96 4E 00 00 B0 4E 00 00	B9 4E 00 00 CF 4E 00 00	-N °N ²N ÌN
000029E0	E3 4E 00 00 F9 4E 00 00	11 4F 00 00 29 4F 00 00	ăN ùN O)O
000029F0	44 4F 00 00 53 4F 00 00	62 4F 00 00 75 4F 00 00	DO SO bO uO
00002A00	81 4F 00 00 99 4F 00 00	2C 4F 00 00 BF 4F 00 00	Œ O ²O -O ¾O

在kernel32.DLL中

2、查找指定函数名称

- 要查找的函数名称字符串为“AddAtomW”，只需要找到RVA数组中第三个元素的值即可（RVA: 4BB3 -> RAW: 3FB3）

3F90h:	52	4E	45	4C	33	32	2E	64	6C	6C	00	41	63	74	69	76	RNEL32.dll.Activ
3FA0h:	61	74	65	41	63	74	43	74	78	00	41	64	64	41	74	6F	ateActCtx.AddAto
3FB0h:	6D	41	00	41	64	64	41	74	6F	6D	57	00	41	64	64	43	mA.AddAtomW.AddC
3FC0h:	6F	6E	73	6F	6C	65	41	6C	69	61	73	41	00	41	64	64	onsoleAliasA.Add
3FD0h:	43	6F	6E	73	6F	6C	65	41	6C	69	61	73	57	00	41	64	ConsoleAliasW.Ad
3FE0h:	64	4C	6F	63	61	6C	41	6C	74	65	72	6E	61	74	65	43	dLocalAlternateC
3FF0h:	6F	6D	70	75	74	65	72	4E	61	6D	65	41	00	41	64	64	omputerNameA.Add
4000h:	4C	6F	63	61	6C	41	6C	74	65	72	6E	61	74	65	43	6F	LocalAlternateCo
4010h:	6D	70	75	74	65	72	4E	61	6D	65	57	00	41	64	64	52	mputerNameW.AddR

此时“AddAtomW”函数名是函数名称数组的第三个元素，数组索引为2。

在kernel32.DLL中 3、Ordinal数组

- 下面查找“AddAtomW”函数的Ordinal值。AddressOfNameOrdinals成员的值RVA: 441C -> 381C, ordinal数组中的各元素大小为2个字节

00003810	DB 8F 00 00 E3 8F 00 00 EC 8F 00 00 00 00 01 00	û ä ì
00003820	02 00 03 00 04 00 05 00 06 00 07 00 08 00 09 00	
00003830	0A 00 0B 00 0C 00 0D 00 0E 00 0F 00 10 00 11 00	
00003840	12 00 13 00 14 00 15 00 16 00 17 00 18 00 19 00	
00003850	1A 00 1B 00 1C 00 1D 00 1E 00 1F 00 20 00 21 00	!

在kernel32.DLL中

4、ordinal

- 将2中求得得index值（2）应用到3中的Ordinal数组即可求得Ordinal（2）。
- `AddressOfNameOrdinals[index] = ordinal (index=2, ordinal=2)`

在kernel32.DLL中

5、函数地址数组-EAT

- 最后查找AddAtomW的实际函数地址，AddressOfFunctions成员的值RVA: 2654 -> RAW: 1A54

00001A50	1C 44 00 00 D4 A6 00 00 05 55 03 00 D9 26 03 00	D Ô; U ù&
00001A60	DF 1C 07 00 A1 1C 07 00 82 93 05 00 66 92 05 00	B i , " f'
00001A70	F9 BE 02 00 F5 8F 00 00 31 23 07 00 1A F6 05 00	ù% ð 1# ö
00001A80	67 59 03 00 42 E4 02 00 19 25 07 00 CA 71 05 00	gY Bă % Êq
00001A90	B0 62 05 00 25 78 05 00 67 68 01 00 E6 CD 06 00	°b %x gh æí
00001AA0	6A CF 06 00 21 CC 06 00 1F CC 06 00 5D 65 01 00	ıî :î ï 1e

在kernel32.DLL中

6、AddAtomW函数地址

- 用4中求得的Ordinal用作索引，可得RVA = 000326D9
- AddressOfFunctions[ordinal] = RVA (ordinal=2, RVA=326D9)
- Kernel32.dll的ImageBase = 7C800000，因此AddAtomW函数的实际地址（VA）为7C8326D9，用OD验证得

7C8326D9 kernel_1.AddAtomW	8BFF	mov edi,edi	
7C8326DB	55	push ebp	
7C8326DC	8BEC	mov ebp,esp	
7C8326DE	FF75 08	push [arg.1]	LoadD1_1.60000000
7C8326E1	6A 01	push 0x1	
7C8326E3	6A 01	push 0x1	
7C8326E5	E8 57D9FDFF	call kernel_1.7C810041	
7C8326EA	5D	pop ebp	LoadD1_1.60001057
7C8326EB	C2 0400	retn 0x4	

实验要求

- 1.使用工具分析PE文件
- 2. 给出程序a.exe，用LORDPE修改它，使之能正常运行，实验结果截图。

提示：

- 1.PE文件判定
- 2.NumberOfSection,此文件有3个节。
- 3.SizeOfOptionalHeader(00E0)
- 4.AddressOfEntryPoint(1000)
- 5.ImageBase(00400000)
- 6.SectionAlinment(00001000)
- 7.FileAlignment(00000200)

实验要求

3.将修改好可以运行的程序a.exe，并使用LordPE，OllyDbg，010Editor等工具对其进行修改，修改后程序可以弹出对话框，内容为姓名+学号，点确定后接着弹出a.exe原来对话框。

- 提示：用LordPE添加新节，对节的大小、存放位置、对齐粒度等进行设置。改后查看a.exe能否被打开，如果不行，则说明该PE文件长度不够，需用010Editor加长a.exe。最后用OLLYDBG对a.exe进行改写，最后需重新设置程序入口点。代码和数据要分别保存



计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

谢谢